МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федерального государственного бюджетного образовательного учреждения высшего образования

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Сопроводительная записка по дисциплине

«Поддержка и тестирование программных модулей»

по теме:

«Создание калькулятора и проведение тестов»

Руководитель

/Берестов Д.Б./

« » 2026

г. Исполнитель

/Муратов Л.А./

« » 2026 г.

Пермь, 2026 г.

ВВЕДЕНИЕ 2
СОЗДАНИЕ РЕШЕНИЯ И ПРОЕКТОВ 3
РАЗРАБОТКА БИБЛИОТЕКИ КЛАССОВ 4
РАЗРАБОТКА WPF-ИНТЕРФЕЙСА 5
МОДУЛЬНЫЕ ТЕСТЫ 7
ЗАКЛЮЧЕНИЕ 8

ВВЕДЕНИЕ

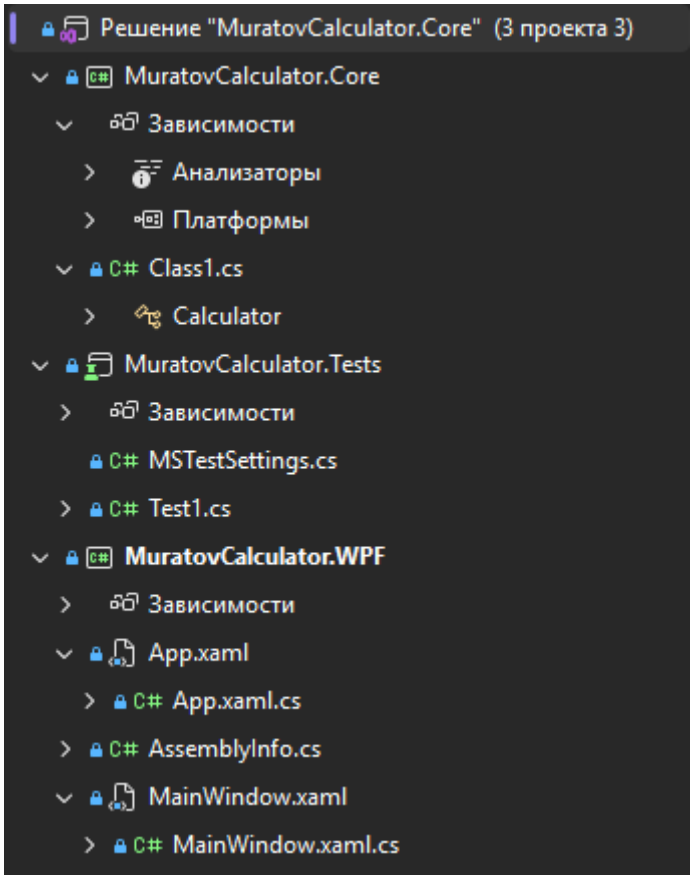
Целью настоящей работы является разработка настольного приложения «Калькулятор» для операционной системы Windows с использованием технологии WPF и языка C#. Приложение должно реализовывать базовые арифметические операции: сложение, вычитание, умножение, деление и возведение в степень.

Дополнительным требованием задания является грамотное разделение приложения на слои: вся вычислительная логика выносится в отдельную библиотеку классов, а пользовательский интерфейс реализуется в виде WPF-приложения, которое к этой библиотеке обращается. Для проверки корректности работы библиотеки необходимо написать набор модульных тестов.

СОЗДАНИЕ РЕШЕНИЯ И ПРОЕКТОВ

Работа начинается с создания пустого решения MuratovCalculator в среде Visual Studio 2022. Затем в него последовательно добавляются три проекта. Первый проект — библиотека классов MuratovCalculator.Core на платформе .NET 8.0; в нём будет храниться вычислительная логика. Второй проект — приложение WPF MuratovCalculator.WPF на платформе .NET 8.0 (Windows); в нём реализуется пользовательский интерфейс. Третий проект — проект модульных тестов MSTest MuratovCalculator.Tests на платформе .NET 8.0; он предназначен для проверки работы библиотеки.

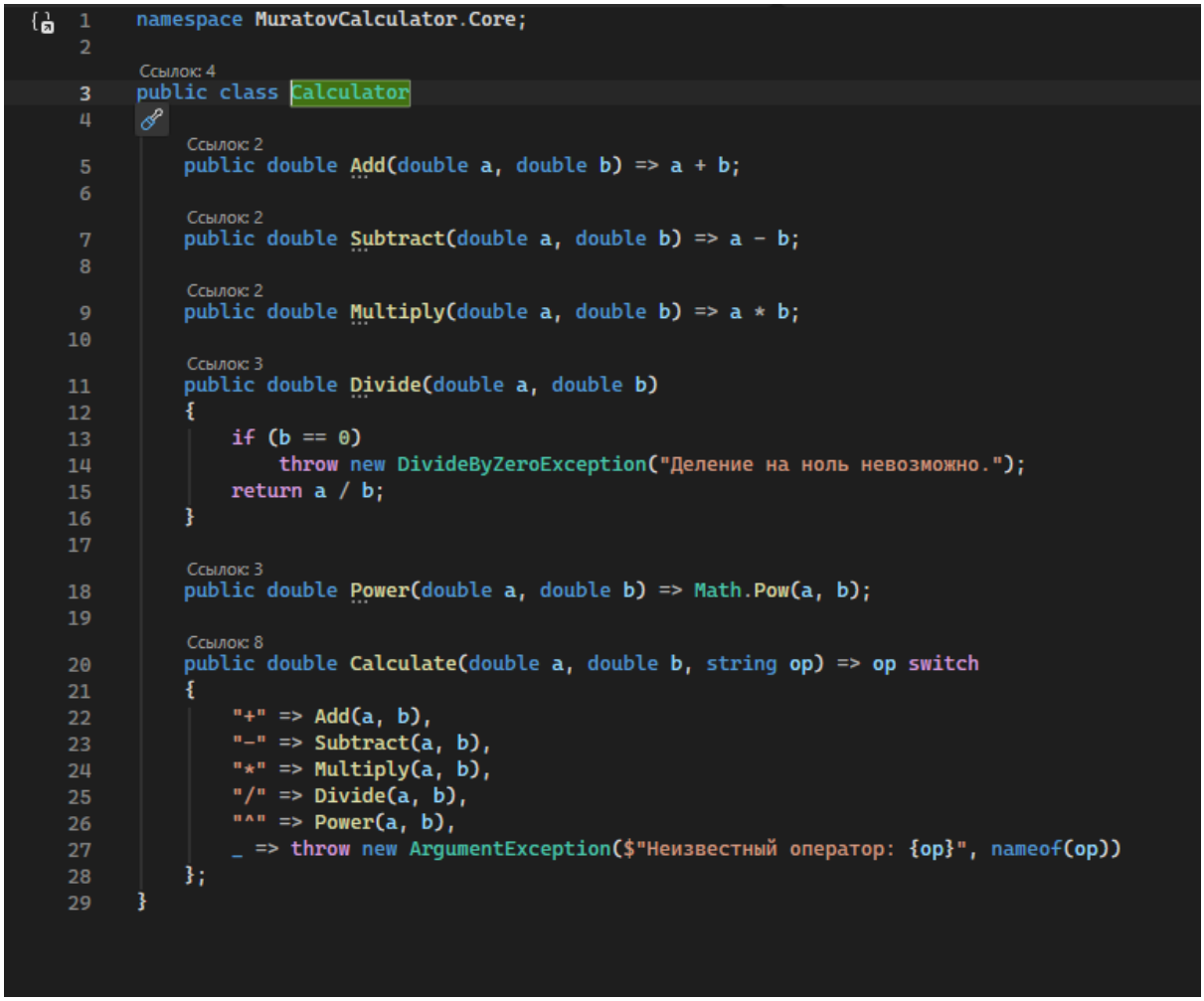
После создания проектов между ними настраиваются ссылки. В проекте MuratovCalculator.WPF через контекстное меню вызывается команда «Добавить → Ссылка на проект», после чего в списке отмечается проект MuratovCalculator.Core. Аналогичная операция выполняется для проекта MuratovCalculator.Tests. В результате обозреватель решений содержит три проекта, а зависимости между ними отображаются в виде раскрывающихся узлов.



(Рис. 1) Обозреватель решений с тремя проектами

РАЗРАБОТКА БИБЛИОТЕКИ КЛАССОВ

В проекте MuratovCalculator.Core автоматически созданный файл Class1.cs удаляется, а вместо него добавляется новый класс с именем Calculator.cs. Класс объявлен публичным и размещён в пространстве имён MuratovCalculator.Core. Внутри класса реализованы пять арифметических методов — Add, Subtract, Multiply, Divide и Power, а также обобщённый метод Calculate, принимающий символ операции и вызывающий нужный метод. Метод Divide проверяет делитель на ноль и в случае ошибки возбуждает исключение DivideByZeroException с понятным сообщением. Метод Calculate при неизвестном операторе возбуждает ArgumentException. Возведение в степень реализовано через стандартный метод Math.Pow. Такой подход позволяет интерфейсу не зависеть от конкретной реализации операций: достаточно передать символ +, -, *, / или ^.



```
namespace MuratovCalculator.Core;

Ссылок: 4
public class Calculator
{
    Ссылок: 2
    public double Add(double a, double b) => a + b;

    Ссылок: 2
    public double Subtract(double a, double b) => a - b;

    Ссылок: 2
    public double Multiply(double a, double b) => a * b;

    Ссылок: 3
    public double Divide(double a, double b)
    {
        if (b == 0)
            throw new DivideByZeroException("Деление на ноль невозможно.");
        return a / b;
    }

    Ссылок: 3
    public double Power(double a, double b) => Math.Pow(a, b);

    Ссылок: 8
    public double Calculate(double a, double b, string op) => op switch
    {
        "+" => Add(a, b),
        "-" => Subtract(a, b),
        "*" => Multiply(a, b),
        "/" => Divide(a, b),
        "^" => Power(a, b),
        _ => throw new ArgumentException($"Неизвестный оператор: {op}", nameof(op))
    };
}
```

(Рис. 2) Файл Calculator.cs

РАЗРАБОТКА WPF-ИНТЕРФЕЙСА

В проекте MuratovCalculator.WPF главное окно приложения описывается в файле MainWindow.xaml. Разметка состоит из двух рядов: в верхнем размещается текстовое поле Display для вывода текущего числа и результата, в нижнем — сетка кнопок UniformGrid с пятью строками и четырьмя столбцами. Кнопки включают цифры от нуля до девяти, знаки арифметических операций, десятичную точку, кнопки очистки C, удаления последнего символа ⌫, смены знака ± и знака равенства. Каждая кнопка имеет собственный обработчик события Click.

Файл MainWindow.xaml.cs содержит класс MainWindow, в котором объявлены приватные поля состояния: первый операнд, символ ожидаемой операции и признак начала нового ввода. Все вычисления выполняет объект класса Calculator, полученный из библиотеки.

Обработчики событий принимают нажатия кнопок, формируют число в поле Display и вызывают метод Calculate в момент нажатия знака равенства. Ошибки, возникающие при делении на ноль или при некорректном вводе, перехватываются и выводятся пользователю в виде всплывающего окна с сообщением.

```xaml
<Window x:Class="MuratovCalculator.WPF.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Калькулятор — Муратов"
        Height="440" Width="340"
        MinHeight="440" MinWidth="340"
        WindowStartupLocation="CenterScreen">
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="70"/>
            <RowDefinition Height="*"/>
        </Grid.RowDefinitions>

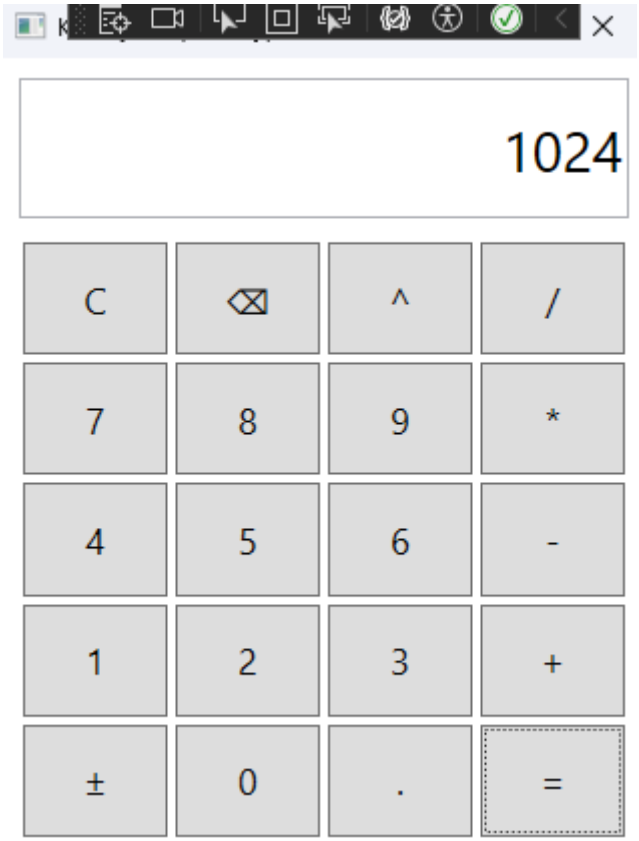
        <TextBox x:Name="Display"
                 Grid.Row="0"
                 FontSize="28"
                 TextAlignment="Right"
                 VerticalContentAlignment="Center"
                 IsReadOnly="True"
                 Text="0"/>

        <UniformGrid Grid.Row="1" Rows="5" Columns="4" Margin="0,10,0,0">
            <Button Content="C"  Margin="2" FontSize="18" Click="Clear_Click"/>
            <Button Content="⌫"  Margin="2" FontSize="18" Click="Backspace_Click"/>
            <Button Content="^"  Margin="2" FontSize="18" Click="Operator_Click"/>
            <Button Content="/"  Margin="2" FontSize="18" Click="Operator_Click"/>

            <Button Content="7" Margin="2" FontSize="18" Click="Digit_Click"/>
            <Button Content="8" Margin="2" FontSize="18" Click="Digit_Click"/>
            <Button Content="9" Margin="2" FontSize="18" Click="Digit_Click"/>
            <Button Content="*" Margin="2" FontSize="18" Click="Operator_Click"/>

            <Button Content="4" Margin="2" FontSize="18" Click="Digit_Click"/>
            <Button Content="5" Margin="2" FontSize="18" Click="Digit_Click"/>
            <Button Content="6" Margin="2" FontSize="18" Click="Digit_Click"/>
            <Button Content="-" Margin="2" FontSize="18" Click="Operator_Click"/>
```

(Рис. 3) Окно MainWindow.xaml

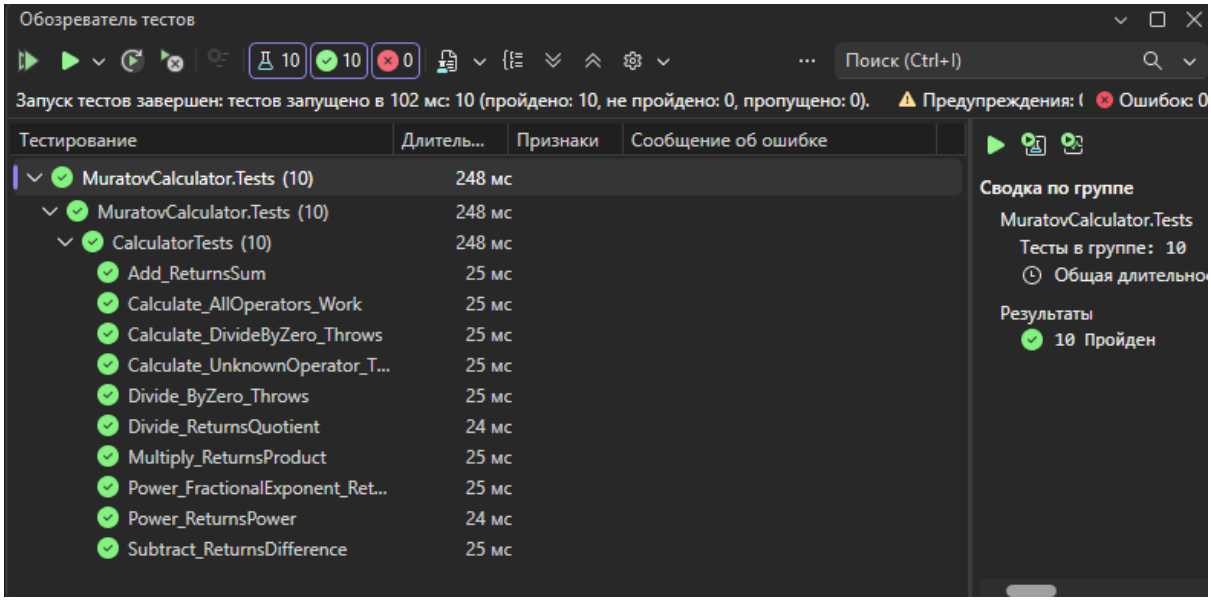


(Рис. 4) Работающее приложение: 2 ^ 10 = 1024

МОДУЛЬНЫЕ ТЕСТЫ

В проекте MuratovCalculator.Tests автоматически созданный файл UnitTest1.cs удаляется, а вместо него добавляется класс CalculatorTests.cs с атрибутом [TestClass]. Внутри класса объявлен экземпляр калькулятора и десять тестовых методов с атрибутом [TestMethod].

Тесты проверяют корректность каждой из пяти арифметических операций, правильность работы метода Calculate со всеми операторами, а также поведение методов при ошибочных ситуациях: деление на ноль должно возбуждать исключение DivideByZeroException, а неизвестный оператор — исключение ArgumentException. Для проверки исключений используется атрибут [ExpectedException].



(Рис. 5) Обозреватель тестов: все 10 тестов пройдены

ЗАКЛЮЧЕНИЕ

В ходе работы было разработано решение MuratovCalculator, состоящее из трёх проектов: библиотеки классов MuratovCalculator.Core, WPF-приложения MuratovCalculator.WPF и проекта модульных тестов MuratovCalculator.Tests. В библиотеке реализованы все требуемые арифметические операции, а также предусмотрена обработка ошибочных ситуаций. Пользовательский интерфейс приложения построен на языке XAML и не содержит вычислительной логики, что соответствует принципу разделения ответственности. Написано десять модульных тестов, покрывающих все публичные методы библиотеки, и все они успешно проходят. Разработанное приложение корректно выполняет арифметические вычисления, устойчиво к некорректному вводу и готово к демонстрации.